Práctica: AulaTech — catálogo de artículos con acceso por roles

Objetivo

Desarrolla una pequeña aplicación PHP que permita consultar un catálogo y gestionar artículos y usuarios según el rol de la persona identificada. Trabajarás con PDO, consultas preparadas con parámetros, sesiones, formularios y control de acceso.

La interfaz estará en español. Utiliza Bootstrap y nombres de variables y funciones en inglés, en camelCase.

1. Preparación y base de datos

Importa `aulatech.sql` mediante phpMyAdmin o el cliente de MySQL. El script crea la base `aulatech`, dos tablas y sus datos. No borra datos existentes: si ya existen las tablas, la importación dará error.

Se proporcionan 20 usuarios y 500 artículos ficticios de material informático. Algunos artículos tienen existencias a cero.

Tabla `users`

Campo	Significado
`id`	Identificador autoincremental
`username`	Nombre de acceso único, máximo 30 caracteres
`password`	Hash de la contraseña, máximo 255 caracteres
`full_name`	Nombre completo, máximo 100 caracteres
`email`	Correo único, máximo 150 caracteres
`role`	`user` o `admin`

Tabla `articles`

Campo	Significado
`id`	Identificador autoincremental
`name`	Nombre del artículo, máximo 120 caracteres
`description`	Descripción
`category`	Categoría, máximo 50 caracteres
`price`	Precio decimal no negativo, con dos decimales
`stock`	Existencias, entero no negativo

Credenciales iniciales

Usuarios	Contraseña	Rol
`admin01` y `admin02`	El mismo texto que su nombre de usuario	Administrador
`user03` a `user20` (ambos incluidos)	El mismo texto que su nombre de usuario	Usuario normal

Ejemplos: `admin01` / `admin01` y `user03` / `user03`.

Las contraseñas se almacenan mediante **hash bcrypt con una sal distinta por usuario**, no mediante cifrado reversible. Esta coincidencia entre usuario y contraseña es exclusivamente para facilitar las pruebas en clase.

2. Inicio de sesión y navegación

La primera pantalla mostrará el formulario de acceso: usuario y contraseña.

- Busca al usuario mediante una consulta preparada con parámetros.
- Verifica la contraseña introducida con `password_verify()` frente al hash almacenado. No compares contraseñas en texto claro ni generes un nuevo hash para compararlo con el existente.
- Si las credenciales son incorrectas, muestra un mensaje genérico y permite volver a intentarlo.
- Si son correctas, regenera el identificador de sesión y guarda únicamente los datos necesarios para identificar al usuario y su rol. No guardes su contraseña ni su hash en la sesión.

- Tras acceder, muestra una bienvenida y el menú correspondiente.

Opción	Usuario normal	Administrador
Ver artículos	Sí	Sí
Ver mis datos	Sí	Sí
Cerrar sesión	Sí	Sí
Añadir artículo	No	Sí
Gestionar usuarios	No	Sí

El menú deberá estar definido en un fichero común e incorporarse con `include` o `require`.

3. Consultar artículos

Muestra una tabla con el identificador, nombre, categoría, precio y existencias de los artículos, ordenados por identificador. El precio se mostrará con dos decimales y el símbolo del euro.

Añade un enlace para consultar la ficha completa de cada artículo, incluida su descripción. Si el identificador no existe o no es válido, muestra un mensaje adecuado.

Como existen 500 registros, implementa paginación de **25 artículos por página**, con enlaces para avanzar y retroceder. Calcula el número de páginas mediante una consulta de recuento. Valida el número de página y vincula `LIMIT` y `OFFSET` como enteros (`PDO::PARAM_INT`).

El administrador tendrá además las acciones **Modificar** y **Eliminar** en cada fila. El usuario normal no dispondrá de esas acciones.

4. Ver mis datos

Muestra el nombre de usuario, nombre completo, correo y rol de la persona identificada. Obtén el registro usando el identificador de su sesión: esta pantalla no debe permitir consultar los datos de otra persona cambiando un parámetro en la URL.

No muestres la contraseña ni su hash. Esta opción es de consulta, no de edición.

5. Añadir y modificar artículos: un único formulario reutilizable

Crea un único fichero, por ejemplo `articleForm.php`, con **una única definición del formulario HTML** para las dos operaciones. No copies el formulario en otro fichero ni dupliques el bloque `<form>` en ramas diferentes.

Petición	Comportamiento
GET sin `id`	Mostrar el formulario vacío para añadir
GET con `id`	Cargar el artículo y mostrar el mismo formulario cumplimentado para modificar
POST sin `id`	Validar e insertar un artículo
POST con `id`	Validar y actualizar el artículo correspondiente

En edición puedes enviar el identificador mediante un campo oculto. Debes validarlo de nuevo en el servidor y comprobar que el artículo existe. El título y el texto del botón cambiarán según la operación.

El formulario tendrá nombre, descripción, categoría, precio y existencias. Para la categoría utiliza un `select` con las categorías del catálogo inicial: Periféricos, Pantallas, Imagen y sonido, Almacenamiento, Conectividad, Redes y Accesorios.

Valida en PHP:

- Nombre y descripción obligatorios; rechaza textos formados solo por espacios.
- Longitudes acordes con los campos de la tabla.
- Categoría perteneciente a las opciones permitidas.
- Precio numérico no negativo, con un máximo de dos decimales y dentro del rango de `DECIMAL(10,2)`.
- Existencias como número entero no negativo dentro del rango del campo.

Si hay errores, muestra el mismo formulario con los datos introducidos y mensajes específicos. Tras guardar correctamente, redirige al listado y muestra un mensaje de confirmación. Actualizar el navegador después de la redirección no debe repetir la operación.

6. Eliminar artículos

Solo el administrador podrá eliminar artículos. Solicita confirmación y envía la eliminación mediante POST. Un enlace GET no debe ejecutar el borrado. Después, redirige al listado con un mensaje de resultado. Gestiona también el caso de un artículo que ya no exista.

7. Gestionar usuarios

Esta opción estará disponible exclusivamente para administradores y permitirá:

1. Listar los usuarios con sus datos y su rol, sin mostrar contraseñas ni hashes.
2. Añadir un usuario indicando nombre de acceso, nombre completo, correo, contraseña y rol.
3. Modificar el nombre completo, el correo y el rol de un usuario. El nombre de acceso se mantendrá fijo durante la edición.
4. Cambiar su contraseña desde la edición: si el campo queda vacío, conservar el hash actual; si se rellena, almacenar un nuevo hash generado con `password_hash($password, PASSWORD_DEFAULT)`.
5. Eliminar usuarios, con confirmación y mediante POST.

Reutiliza también **una única definición del formulario de usuarios** para alta y edición. La regla de formulario único se aplica a cada entidad: habrá un formulario de acceso, uno de artículos y uno de usuarios; los pequeños formularios de confirmación de borrado o cierre de sesión son independientes.

Valida campos obligatorios, longitudes, correo válido, rol permitido y ausencia de nombres de usuario o correos duplicados. En alta, la contraseña es obligatoria; para esta práctica limita su longitud a entre 8 y 64 bytes. Las cuentas iniciales se mantienen con las credenciales facilitadas, aunque sean más cortas.

Para simplificar la gestión de sesiones, **el administrador no podrá eliminar su propia cuenta ni cambiar su propio rol**. Comprueba esta restricción en el servidor. Para las demás peticiones protegidas, consulta el usuario actual en la base de datos: si su cuenta ha sido eliminada, invalida su sesión; si su rol ha cambiado, aplica los permisos actuales.

8. Cierre de sesión

El cierre se realizará mediante POST. Vacía los datos de sesión, elimina la cookie de sesión e invalida la sesión. Redirige al acceso. Después de cerrar sesión, una nueva petición a una pantalla protegida deberá requerir identificación.